低功耗离线全国矢量地图导航自行车码表（Helm One）· 技术报告

1、信息表

项目	内容
作品名称	低功耗离线全国矢量地图导航自行车码表
队伍名称	第九个下弦月
队长姓名	金松超
联系电话	17858385203
仓库链接	https://github.com/open-vela/contest2026_048_dijiugexiaxianyue
团队分工	金松超（队长）：自研板与 BSP/驱动、离线地图工具链与固件、手机 App、测试与文档，本作品由队长一人独立完成。
选题方向	AI 硬件产品创新 + 新硬件平台适配（本项目既做 AI 硬件产品创新，也完成了全新硬件平台的驱动适配与整机集成）

2、摘要

1. 车把正对太阳：电容屏反光、戴手套、下雨都点不准，低头伸手更危险；手机支架耗电、没网就没图；成品码表贵且闭源。本作品用 openvela + SF32LB52 做出可上路的开源码表 Helm One：半透半反无触摸屏 + 双实体键，全国路网切成 3 km 网格存于 microSD 按需加载，实现离线规划、GPX 导航与返航。围绕低功耗重写 LCD、SD、GNSS、蓝牙双角色与 MTP 驱动：LittleFS 挂载 0.474 s，SD 顺序读 1.0 → 3.1 MB/s，三城离线地图约 314 MB。自研 PCB 与外壳、固件、App、后台服务全链路齐备，软件开源（Apache 2.0）。

3、正文

3.1 绪论

2. **项目背景与问题定义：**应用场景是公路与通勤骑行，码表固定在车把上、正对太阳。真实痛点有四条：① 电容触摸屏在户外反光、戴手套、下雨时都点不准，还迫使骑手低头伸手；② 手机支架耗电、分心，且没有信号就没有地图；③ 成品码表价格高、固件闭源、生态锁死；④ 为在阳光下看清，普通 IPS 必须把背光拉满，功耗高且画面仍然发灰。

3. **目标受众：**公路与通勤骑行者，以及希望在 openvela 上开发运动穿戴设备的开发者。

4. **技术难点：**本项目要攻克的关键挑战集中在四处。

5. **（1）双键无触摸交互。**整机只有 KEY1、KEY2 两颗按键（另有电源键），却要覆盖 7 个主页面、28 个菜单屏，以及记录、导航、自动/手动暂停、保存确认等状态迁移，且不能在菜单打开时误触发记录。

6. **（2）在 MCU 上跑全国离线矢量地图。**要在 16 MB PSRAM 的 MCU 上承载全国路网，必须解决分片方案、按需加载、内存上限与渲染时序；跨网格路由的接缝处理尤其困难，实测曾出现绕行 3576 m（实际最短 1594 m）的退化。

7. **（3）外设驱动的稳定性。**SDIO 控制器会卡死在 CMD_BUSY 状态导致整卡假死；LittleFS 在已写满 14 GiB 的卷上首次分配需约 24 s，会直接触发看门狗复位；GNSS 模组会“静默死亡”（写操作静默失败、无错误返回）。这些都必须在驱动层设计针对性的检测与恢复链。

8. **（4）单射频蓝牙双角色与共存。**设备既要作为 GATT Server 连接手机，又要作为 GATT Client 连接心率带、踏频器与功率计；而广播与扫描在单射频上互斥，还要与 GNSS、SD 的时序相互竞争。

9. **创新点：**技术、场景与交互三个层面各有差异化。

10. · **交互层面——把“精细操作”移出车把。**整机不做触摸屏，用半透半反屏 + 两颗实体键完成车上全部操作：短按翻页、长按菜单或开始/暂停，戴手套、湿手、雨天都能用，日光越强对比越清楚；需要点地图、画路线、分享的动作放到手机 App。这是对码表交互范式的重新划分，而不是把手机界面缩小。

11. · **技术层面——MCU 上的全国离线矢量地图运行时。**全国路网切成 3 km 网格、按当前位置按需加载；保留画布 + 脏矩形局部重绘 + 每次 tick 8 ms 的增量渲染预算，使 240×320、无 GPU 的 MCU 也能稳定跟车绘制与转向提示。

12. · **场景层面——完全离线的独立整机。**记录、离线路网规划、沿 GPX 导航、返航都在车上完成，不依赖手机与网络；手机断连不影响骑行，板载定位未锁定时才用手机位置临时顶上。

13. · **工程层面——可复用的开源整机与板级约定。**系统、板级、骑行应用、手机 App、地图工具链全部开源（Apache 2.0），并把户外色板、网格按需加载、蓝牙双角色等经验整理为可供社区复用的板级约定。

3.2 系统方案设计

14. **系统总体架构：**整机分为三层，如图 1 所示。输入层负责获取运动与位置信息：板载 GNSS（u-blox MAX-M10S）提供位置、时速、航向与海拔，手机通过 BLE 特征 0xFF14 推送辅助定位，心率/踏频/功率计通过 BLE 连接，IMU（BMI270）与气压计（BMP388）提供姿态与短时高程，两颗按键提供唯一的人机输入。

15. 核心层运行在 openvela（NuttX）之上，包含位置与速度融合（飞点抑制、海拔融合）、骑行记录（轨迹环 + 1 Hz GPX）、离线路径规划（Dijkstra + 跨网格拼接）、整机状态机，以及 LittleFS/FAT 存储栈。输出层把结果同时送往 240×320 半透半反屏（LVGL 界面 + 离线矢量地图）与手机 App；App 反向下发路线 GPX 与 OTA 固件。

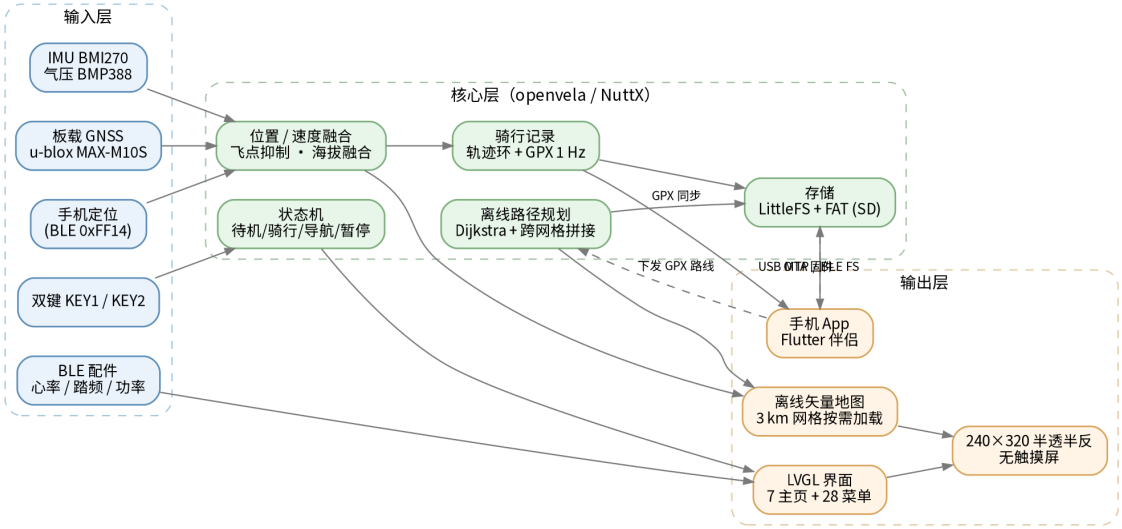


图 1　系统总体架构与数据流

16. **模块组成与数据流要点：**传感器/配件数据单向流入核心层；核心层的记录与地图数据经 USB MTP 或 BLE FS 交给手机与电脑；手机只承担“需要手指与网络”的动作（下发路线、上传星历、OTA、通知同步），不参与车上的实时闭环。

17. **开发板与系统选型理由：**① 平台：选用 SiFli SF32LB52 系列（量产目标 SF32LB527UD6，16 MB PSRAM、片内 PMU 与充电）。该芯片把蓝牙核与应用核分离，低功耗路径已在手表、手环等亿级设备上验证。② 操作系统：选用小米开源的 openvela，而不是自写私有 RTOS——LVGL、FreeType、VFS、蓝牙框架（zblue + OpenVela BT Framework）、USB device、CMake 构建体系都可直接复用，使单人能在数月内完成从驱动到 UI 的整机实现。

18. **与备选方案对比：**ESP32 系列缺少与量产 MIP 屏匹配的 QAD-SPI + 显示控制器通路，且大屏刷新与功耗不利；STM32 + FreeRTOS 方案需要自行搭建图形、蓝牙协议栈与文件系统生态，工作量与风险显著更高；Linux 方案（如树莓派类）功耗与成本都不适合一台靠小电池连续工作十几小时的码表。综合结论是 SF32LB52 + openvela 在“低功耗 + 半透半反无触摸 + 双键 + 离线图”这一组合上最匹配。

19. **端 / 云职责划分：骑行功能不依赖云端，但本作品包含一个自建后台服务，交付物为自研硬件（自研 PCB + 自研外壳）、固件、手机 App、后台服务四块。① 码表端在车上完全独立：记录轨迹、离线路网规划、沿 GPX 导航、把本次记录反向作为返航，全部不依赖手机与网络；② 手机 App 端承担「需要手指或网络」的三类工作：地图交互（点选途经点、绘制/导入 GPX，在线吸路后下发）、需要网络的资源（下载当天广播星历并注入、检查并下载 OTA 固件）、手机的富交互（通知转发与图标上传、骑行分享）；③ 自建后台服务（ota.jinsc.top）：只承担 OTA 升级清单与固件分发（见「(五) 后台服务」），不参与骑行闭环——离线、断网时整机功能完全不受影响。**

20. **断网 / 弱网降级策略：**① 手机蓝牙断开：车上记录、离线导航、沿 GPX 导航、返航全部照常，重连后补齐通知、文件与 OTA；② 板载定位未锁定：仅在锁定前使用手机推来的位置，板载锁定后仍以板载为准，手机位置停推 5 s 即撤销；③ 卫星质量差或静止：抑制飞点，短时失锁保持上一拍时速而不跳变为 0；④ 地图存储被占用：插入 USB MTP 时地图让出存储，拔线后继续使用；⑤ 单路配件掉线：界面保留上次有效值，不立即清零，可配置自动回连。

21. **（一）蓝牙与手机协同模块。**蓝牙采用双角色：对手机是 GATT Server（自定义服务 0xFF10），对心率/踏频/功率计是 GATT Client。自定义 Companion 协议当前版本 0x0008，

包含 12 条特征（0xFF11–0xFF1A）：协议版本、设备状态、控制、GNSS 定位注入、文件系统命令与数据、通知、轨迹点、导航路线、设备信息。实测 MTU 协商为 193（有效载荷 190 B），文件传输稳定在约 15 KB/s，支持 CRC32 校验与断点续传。单射频上，广播与扫描按时间片轮转（配对窗口 200 ms / 已配对空闲 1280 ms / 未配对且不在窗口内不广播），并让位于 GNSS 的休眠与星历注入。

22. **（二）文件传输与固件升级模块。**考虑到车上没有触摸屏、大文件无法点选，提供三条路径：① USB MTP——电脑把码表识别为媒体设备，产品固件只暴露 /mnt/lfs/mtp；② BLE FS——出门不带线时传路线、GPX 与小文件；③ OTA——App 把新固件写入升级槽，下次开机自动切换，旧槽可回退。

23. **（三）存储与启动模块。**microSD 上用分区表划分：ftab 与二级 boot（160 KiB）、产品固件槽 main（4 MiB）、工厂固件槽 factory（4 MiB）、KV 区（256 MiB，含 OTA 固件目录 /fw）、用户文件系统 FS_REGION（512 MiB，LittleFS），余量给 FAT 区存放全国地图与矢量字体（产品只读）。自研二级 boot（2SFBL）按 KV 中的 persist.boot.target 选择启动目标，默认链为 /fw → main → factory。

24. **（四）多设备角色与分工。**码表是唯一带屏的主节点，负责采集、记录、规划与显示；手机是配置与内容源（路线、星历、通知、固件）；心率带、踏频器、功率计是被动的 BLE 外设，仅提供服务数据。三者之间没有对等协同，因此不存在跨设备时钟同步问题，协议全部为单向拉取/推送。各节点运行的系统：码表运行 openvela（NuttX 内核）与自研板级固件；手机端运行自研 Flutter App（Android / iOS）；心率带、踏频器、功率计为各厂商成品外设，运行各自固件，只提供标准 GATT 服务。

25. **（五）后台服务（自建，OTA / 固件分发）。App 以 GET https://ota.jinsc.top/ota/helm-one/firmware.json?token=…&hw=<硬件版本> 拉取升级清单：token 鉴权、按硬件型号过滤，避免把不匹配的固件推给旧硬件；清单给出固件版本与下载地址，App 下载后校验 SHA-256 与长度，再经 BLE 写入设备的固件槽（/mnt/kv/fw），由自研二级 boot 在下次开机完成切换并保留上一版本回退。该服务只做分发，与骑行功能解耦；后台代码与 App 同仓维护。**

3.3 核心算法与技术原理

26. **关于 AI 算法的说明（如实陈述）：本作品未使用端侧或云端的神经网络模型：没有集成 TFLite / CMSIS-NN / NPU 推理，也没有语音唤醒功能（因此不涉及“你好，openvela”唤醒词）。本作品落地的 openvela 核心能力是“图形”。与 AI 相关的工作集中在 AI-Native 开发流程本身（详见 3.6 节：ZCode + DeepSeek-V4-Flash 结对开发、mimosa 钩子式代码审查）。本节说明产品侧的算法贡献：离线路由、地图渲染与多源传感器融合。**

27. **模板所询的云端模型：本作品未使用（包括大赛 MiMo）——产品侧不含端侧或云端神经网络推理，骑行链路全部是确定性算法（见本节 (1)~(5)）；AI 相关工作在开发流程侧，详见 3.6 节。**

28. **（1）离线路由：堆式 Dijkstra + 跨网格 portal 拼接。**路网以 VGRF 格式存于 SD，包含节点、无向边、字符串表、逐节点海拔与 SNAP 网格。实测规模：滁州 338 282 节点 / 915 902 边，南京 415 669 节点 / 1 108 600 边，广州 1 090 214 节点 / 3 562 467 边。

29. · 吸附：当前位置在 150 m 内取最多 8 个候选边，候选池 800 m；超过 4 000 m 判定为出图。

30. · 规划：堆式 Dijkstra（dijkstra_heap_push / pop），每约 4 ms 让出一次以便中断；worker 线程 15 s 卡死检测，最多自愈 3 次。

31. · 跨网格：走廊规划最多跨 4 个区域、16 条边界、4 次 portal 尝试；portal 匹配阈值 120 m（回退 150 m），拼接去重 5 m。该处曾出现绕行 3576 m 而实际最短 1594 m 的退化，修正后按区域走廊逐段拼接。

32. · 导航判定：偏航 35 m 触发重规划，并加 8 s 最小间隔防止“重规划风暴”（曾观察到 1 s 内重规划 2–3 次）；到达判定 45 m，速度大于 3 km/h 时自动退出导航。

33. · 容量约束：规划折线上限 2 048 点，超出整条路线被拒（而不是静默截断）；转向提示表 64 条；途经点上限 32 个。

34. **（2）GPX 轨迹导航的滑动窗口。**沿 GPX 导航不一次性把整条轨迹读入内存，而是滑动加载：每次约 8 km，剩余不足 2.5 km 时续载下一段，段间保留 400 m 重叠；起点衔接允许 1 000 m、途中衔接允许 2 000 m。返航即把折线反向，并把横向走廊从 30 m 放宽到 100 m，以容忍对向车道的偏移。

35. **（3）地图渲染：保留画布 + 脏矩形 + 增量预算。**地图不逐帧解码瓦片，而是维护一块比屏幕大一圈（外扩 192 px）的 RGB565 保留画布：小位移直接把画布平移，走出边距才分片补边。每次定时器最多处理 2 个分片、预算 8 ms，避免阻塞 UI 线程与看门狗。脏矩形外扩 24 px、单次最多 8 个；道路绘制做抽稀（最小段长 6 px、最大偏差 2.8 px）；路名与地名放在独立的透明标签层上，不烘焙进底图，标签上限 256（其中道路标签 96）。分片文件格式为 VPKG 包 + RIDX v2 索引（52 B 头 + 每个分片 16 B 记录），单个分片上限 128 KiB。

36. **（4）海拔与坡度融合。**海拔以路网 DEM 高程为主体、气压计跟短时起伏：DEM 逐节点高程覆盖率实测 100%，气压换算用 ISA 常数 44 330.77；DEM 偏置用 EMA（α=0.05）缓慢收敛，参考气压用 α=0.08 更新。累计爬升设 3 m 迟滞，停车或速度过低时不计入，避免气压噪声累积成虚假爬升。坡度来自 IMU 俯仰角：一阶低通 τ=1.2 s、限斜率 8 %/s、显示钳位 ±40%，并支持水平放置后归零。

37. **（5）速度解算与飞点抑制。**速度源改用 u-blox UBX-NAV-PVT 的 gSpeed（多普勒速度）与 sAcc（精度），替代原始 NMEA 的 SOG 字段，精度与静止判定都更可靠。静止门采用迟滞：进入静止 3 m、退出静止 8 m、保持 4 s；静止时速度归零，短时失锁则保持上一拍速度，避免显示在 0 与正常值之间跳变。

38. **落地了哪几项 openvela 核心能力：**大赛要求落地“图形 / AI / 多媒体”中的至少一项，且使用 openvela 系统能力（NuttX 内核仓库除外）。本作品落地的是【图形】：LVGL 9 骑行界面 + 自绘全国离线矢量地图 + FreeType 矢量字体渲染；同时使用了 openvela 自有的蓝牙框架与 KVDB 框架，二者均不属于 NuttX 内核仓库。

39. **用到的 openvela 组件与配置证据：**① 图形：apps/graphics/lvgl（CONFIG_GRAPHICS_LVGL=y）+ external/freetype（CONFIG_LV_USE_FREETYPE=y），承载界面、地图画布与矢量字形渲染；② 蓝牙框架：frameworks/connectivity/bluetooth（CONFIG_BLUETOOTH_FRAMEWORK=y、CONFIG_BLUETOOTH_SERVICE=y）+ external/zblue 主机栈（CONFIG_BLUETOOTH_STACK_LE_ZBLUE=y），双角色 BLE 与 Companion 协议都建立在这套框架之上；③ KVDB 键值框架：frameworks/system/utils/kvdb（CONFIG_KVDB=y，CONFIG_KVDB_PERSIST_PATH="/mnt/kv/db"），承载蓝牙配对信息、设备控制开关与开机槽标记；④ 文件系统与构建：VFS 与 LittleFS/FAT，配合板级 vela_override 抽换机制（CMake 从库中移除上游 .c 并换为板级实现）；⑤ 运行时 Skill 规范：packages/ai_agent 的 /data/agent/skills/ 约定，见 3.4 节。

40. **为什么必须用矢量字体：**全国地名需要数千个汉字字形，位图字库塞不进 Flash。因此从全国

OSM 命名标签中提取字符集（广州抽样的 660 587 条命名标签中提取出 3 671 字，其中 CJK 3 348 字），经繁简转换后生成 MiSans 子集 TTF，由 FreeType 在固定字号下栅格化；菜单与标题仍保留 4bpp 位图字模，避免每帧重复栅格化的开销。

41. **对 openvela 的优化、拓展与改进建议：**一是用 vela_override 抽换上游 .c 文件（如 fs_procfs_mount.c 的 4 GiB 溢出、fs_fat32util.c、usbdev/mtp.c、bluetooth/id.c 等共 16 个），并在链接脚本中放置标记以验证替换确实进入镜像；二是向社区提交补丁，例如 zblue 的 id.c 允许在已有连接存在时忽略 LE Set Random Address 返回的 -EACCES（否则无法同时"连手机 + 扫传感器"）、scan.c 的 scan_ctx 空指针保护、HCI H4 接收重开的有界重试等；三是建议把"户外配色板、全国网格按需加载、蓝牙双角色时序、大文件占用存储时界面让路"沉淀为可复用的板级约定。

3.4 系统实现

42. **软件 / 固件架构总览：**本作品没有改动 openvela 上游仓库，而是在 vendor/HelmOne/ 下建立了一棵与官方 vendor/sifli/ 平行的自研 vendor 树，配合 boards/sf32lb52/helmone/ 板级目录。代码规模如下（不含 vendored LVGL 与生成字模）：

43. · chips/sf32lb52/：32 个 .c、约 2.4 万行。自研芯片层胶水，含 DVFS、SDIO、独立看门狗、LittleFS super、蓝牙适配器与 H4、USB device、LCPU 启动、CPU 负载统计等。

44. · boards/sf32lb52/helmone/：233 个 .c、约 23.3 万行（其中约 9.3 万行为生成的位图字模）。分为 bsp/（板级初始化与引脚）、ctl/（系统控制与设备控制）、domain/（GPX 编解码与记录）、services/（ble_companion、mtp_simple、GNSS、LCD 显示服务）、ui/bicycle/（lvgl_page 页面、vmap 地图引擎、framework 框架）、nsh/（30 余个机上探针与自测命令）、vela_override/（上游抽换）。

45. · boot_loader/：121 个 .c、约 6.1 万行。自研二级 boot（2SFBL）：UART msh、SGL 引导界面、KV 持久化、OVNX 镜像加载与 CRC 校验、分区表生成。

46. · 手机 App：Flutter（107 个 .dart、约 3.3 万行）。

47. · 合计自研代码约 25.7 万行；此外以 vela_override 方式替换了 16 个上游源文件。

48. **关键技术子系统的实现：**① 电源与频率：DVFS 设 72 / 96 / 144 / 240 MHz 四档（分属 S0 / S1 电压域），每 250 ms 采样 CPU 负载，降档需连续稳定 2 s，交互触发 1 s 的 boost；切换主频时必须对 SD 做"停—改频—重启"三明治操作，否则会打断 SD 传输。② 可靠性：独立看门狗 15 s 超时、2 s 喂狗、10 s 心跳，且要求空闲线程近期运行过且 UI 线程进过循环，从而只在整机真死时复位；diag 线程每 500 ms 巡检各子系统，按分级阈值（蓝牙 4/20 s、GNSS 12/30 s、DVFS 2/10 s、文件系统 3/15 s）上报并踢活停顿线程。③ 日志：非阻塞的 2 KB RAM 环形缓冲，只落 WARNING 以上，崩溃时随 coredump 一并写出，避免在故障时刻反被写盘拖死。

49. **启动链（图 2）：**上电后片内 Mask ROM 提供约 1 s 的 UART 调试窗口，并从 SD 的 ftab 分区把 2SFBL 拷入 SRAM 0x20020000；2SFBL 读取 KV 中的启动目标，挂载文件系统并驱动屏幕显示引导进度；随后按 /fw → main → factory 的顺序选择固件。OTA 固件存放在 KV 的 /fw 目录（最多保留 12 个），按文件名 X.Y.Z 排序、同版本再比较 OVNX 头的构建时间，逐个做 CRC32 校验，第一份通过者胜出。OVNX 镜像布局为 1 KiB 文件头 + payload + 4 B CRC32，加载时跳过文件头、按 2 KiB 分块搬入 PSRAM，全部载入后再校验一次。工厂固件与产品固件同址但互斥，任一时刻只运行一个。

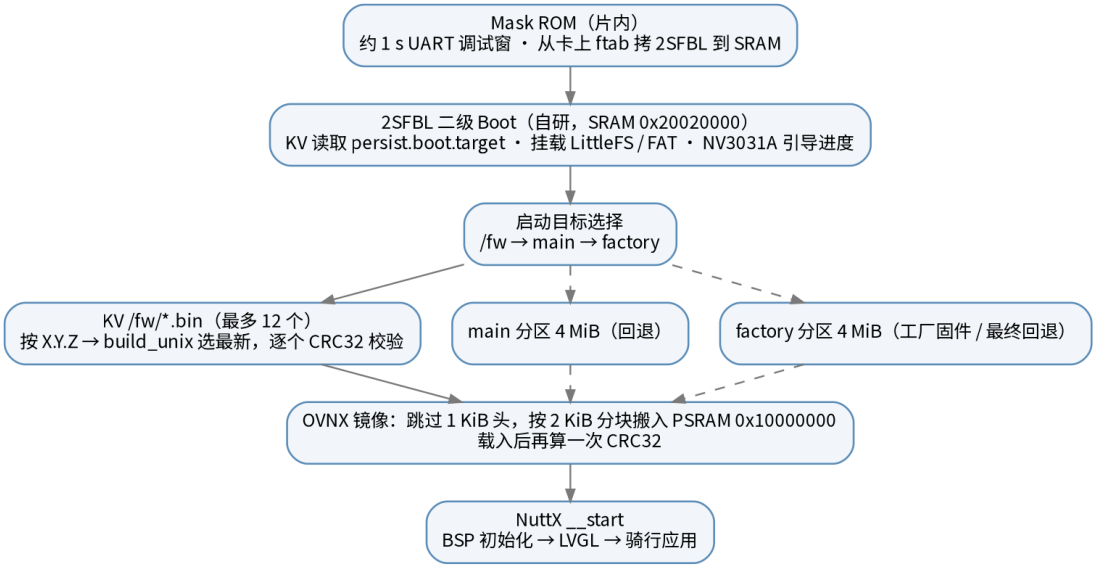


图 2　启动链：Mask ROM → 2SFBL → OVNX → NuttX

50. **骑行记录与导航流程（图 3）：**定位数据进入后先做有效性与速度滤波，再写入轨迹环（点距 3 m、约 5 万点，约合单圈 150 km），同时以 1 Hz 写入 GPX 文件；上屏折线经过抽稀，最多 384 个顶点，并按整公里打标（150 个槽位）。若处于导航态，则同时监控偏航与到达；静止条件满足约 4 s 进入自动暂停，恢复运动约 1.5 s 自动继续，手动恢复后 1 分钟内不再自动暂停；长按 KEY2 进入暂停条，可保存或丢弃。

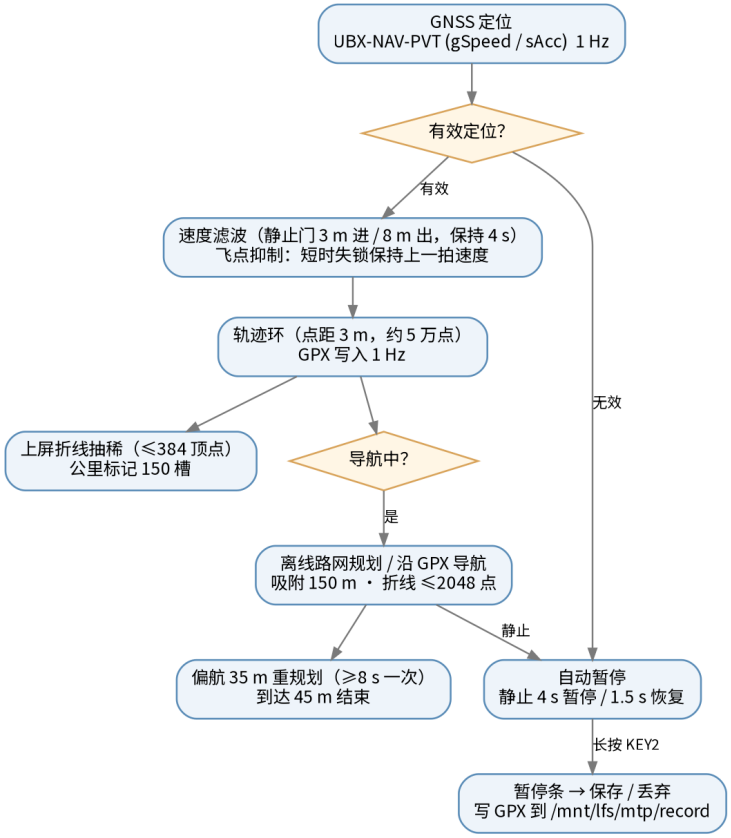


图 3　记录与导航的关键流程

51. **是否完成全新硬件平台适配或驱动开发：是。**本作品在同一颗 SoC 上建立了自研 vendor 树与自研板级目录，并新写了显示、存储、定位、蓝牙、USB 等多类驱动，而非仅在官方例程上改应用。硬件侧同样为自研：自研 PCB 与自研结构件（外壳）均已完成并用于整机验证，作品展示照片中的整机即为自研板 + 自研外壳；硬件、固件、App、后台服务与 PC 端地图工具链全部由本队独立完成。

52. **芯片与整机硬件：SiFli SF32LB52 系列（量产目标 SF32LB527UD6，16 MB PSRAM、片内 PMU 与充电）；整机为自研 PCB + 自研外壳，全部功能验证、稳定性测试与户外实骑都在该自研硬件上完成（见作品展示照片 01–06）。**

53. **外接模块、接口与关键 BOM：**

54. · 显示：NV3031A-AI，240×320 RGB565，4 线 QAD-SPI @24 MHz，2.4 英寸半透半反无触摸屏，背光 PWM（PA01）、面板电源（PA10）；未引出 TE 信号。

55. · 定位：u-blox MAX-M10S（UART1，38400 8N1，1 Hz 输出），支持 UBX-NAV-PVT 与 NMEA，MOD 电源控制（PA43），配外置天线。

56. · 运动传感：BMI270（IMU）、BMP388（气压）、MMC5983MA（磁力计），均走 I2C。

57. · 存储：microSD（SDMMC1，4-bit，卡时钟上限 24 MHz）+ 片外 Flash。

58. · 电源：ETA9184 充电/电量计（I2C），电池电压经 GPADC 采样，扬声器 PA 使能（PA35）。

59. · 人机：KEY1（PA30）翻页、KEY2（PA33）确认、电源键（PA34）。

60. **驱动类型、适配难点与解决方案：**

61. · LCD：官方例程仅有 ST7789S，本作品新增 NV3031A 驱动；该面板没有引出 TE 线，因此改为依靠 LCDC 的 PUTAREA 完成中断，并配套超时/中止/恢复状态机（慢帧阈值 80 ms、中止宽限 150 ms、恢复冷却 8 s），解决了无 TE 时难以判断硬件是否仍在出图的问题。

62. · SD：ExtDMA 属内存到内存拷贝、不能直接喂 SD FIFO，PIO 下又需要读取未公开的寄存器判断水位，因此读写都必须走 DMA。读改为 CMD17/CMD18 + DMA，写按“DMA 内存→FIFO、发命令、等卡应答后再 DATA_START”的顺序实现。超时先发 CMD12 + CMD13 回到 TRAN 状态再重试，并禁止在 DATA 态发 CMD8（否则一次超时会被放大成整卡重识别）；控制器卡死在 CMD_BUSY 时用模块级 RCC 复位救回；同时对 4 GiB 地址回绕做钳位，避免读卡时覆盖 boot 镜像。

63. · GNSS：模组会“静默死亡”——写操作没有错误返回但不再产出数据。恢复链为：静默 5 s 重开 UART，连续两次不恢复则断电 2500 ms、上电后等待 1000 ms 稳定；室内持续搜星时不对模组反复断电，星历注入期间避免二次打扰。

64. · 蓝牙：单射频需要同时承担对手机的 GATT Server 与对传感器的 GATT Client。SF32 的 LCPU 在已有广播/扫描/连接时禁止设置随机地址并返回 -EACCES，导致无法“连手机的同时扫描传感器”；通过向上游 zblue 提交补丁、忽略该错误并复用当前身份解决，并补齐 scan_ctx 空指针等防御。

65. · USB：自研 MTP 应答器（替代原 GLib 方案）并对 NuttX 的 USB device 层做板级抽换，产品固件只暴露骑行目录，工厂固件才暴露三卷并可见固件槽。

66. **车机 UI（LVGL）：**主界面共 7 页：待机、概览、心率、速度、地图、转向、爬升，其中转向页仅在导航时出现。数据页约每 3 秒轮换一组四项（时间/里程/均速/极速、心率/踏频/功率/圈

数、海拔/坡度/爬升/下降），并各自带走势；地图类页面的底栏同样约 3 秒轮换两项。菜单共 28 屏，覆盖导航、蓝牙设备、记录、设置、测试、星历、常用点、收件箱、工具箱与系统状态等。配色针对半透半反屏做提亮处理，提供日光与夜间两套主题与亮度百分比。

67. **手机 App（Flutter）：**App 分三栏——骑行、设备、设置。骑行栏负责会话记录、GPX 播放与图表、周目标统计、海报分享、途经点与收藏；设备栏显示电量、记录状态、定位来源、配件连接与剩余存储，并提供 OTA、coredump/日志抓取、文件浏览与星历页面；设置栏包含中英文、保活、离线城市地图下载、缓存与日志导出、App 自更新。

68. **App 侧关键实现：**

69. · 路线规划：点选/绘制途经点后，先经在线服务把落点吸附到道路，再按路网补齐段间空隙，最后导出 GPX 并通过 BLE 下发；码表侧只做离线执行。

70. · 通知转发：App 通过 NotificationListenerService 转发消息，按应用白名单与锁屏/骑行条件过滤，并将应用图标按 MD5 增量同步到车上（车上按文件名拉取缺失图标）。

71. · 星历：App 从网络拉取当天广播星历，转换为 UBX-MGA 消息后写入码表并立即注入模组，无需重启；连接期间每 2 小时同步一次。

72. · OTA：App 下载固件并校验 SHA-256 与长度后上传到固件槽，版本号与硬件型号双重校验，不匹配即拒绝。

73. **沉淀的自定义 Skill：vela-helm-one。**运行时路径：/data/agent/skills/vela-helm-one.md（单文件）。这是一个面向“openvela + SF32LB52 骑行整机”的板级知识与操作 Skill，把散落在仓库文档中的事实、约束与故障恢复手册固化成可被 AI 助手直接调用的能力。格式遵循 openvela packages/ai_agent 的 Skill 规范：首行 # 标题，第二行起一段描述（标题与描述会被拼接进系统提示，作为常驻索引），其后是 When to use / How to use / Example 三节正文，正文由助手按需读取。

74. **与运行时机制的配合：**openvela 的 ai_agent 会在启动时扫描 /data/agent/skills/ 下的扁平 .md 文件，把每个 Skill 的标题与描述组织成系统提示中的索引行，正文则在命中时用 read_file 读取；目录的文件名、大小与 mtime 会参与哈希，改动后无需重启即可热重载。本 Skill 已按该约定落地为一枚扁平 .md 文件，可直接拷入 /data/agent/skills/ 生效；也可直接对助手说“把这次的处置写成一个 Skill”，由内置的 skill-creator 在运行时生成同一格式的文件。

75. **定义与触发场景：**

76. · 触发场景一——板级/驱动开发：在 openvela 上新增或修改 SF32LB52 的 LCD、SD、GNSS、蓝牙、USB 驱动时，Skill 提供引脚、分区、时钟与阈值的权威值，避免与既有实现冲突。

77. · 触发场景二——地图工程：生成或调整全国地图分片、修改路由阈值与吸附半径时，Skill 提供网格规则、文件格式与参数约束，并提示同步更新文档口径。

78. · 触发场景三——协议与 App：修改 Companion 协议时，Skill 强制校验双端版本一致性（固件 companion_proto.h 与 App companion_proto.dart），并给出特征号分配与兼容策略。

79. · 触发场景四——稳定性排查：出现看门狗复位、SD 假死、GNSS 静默、蓝牙断连、界面卡顿时，Skill 直接给出已知根因与处置步骤。

80. **Skill 内容组成：**① 权威值速查（分区偏移、引脚、时钟上限、界面与阈值口径、地图网格与协议要点）；② 强制约束（改阈值先改 system_states.md、AI Coding 日志不得手工删改、协议版本必须双端同步、不改上游 nuttx/ 而走 vela_override、烧录与调试必须走指定工具

链）；③ 故障速查表（现象 → 根因 → 处置，覆盖 SD CMD_BUSY 复位、DATA 态禁 CMD8、lfs_alloc 首次分配导致开机复位、静止假速度、蓝牙双角色 -EACCES、看门狗误触发判定等）；④ 机上探针清单（ptab / bootinfo / df -h / free / ps 等 run_shell 用法）。其完整定义已随本报告一并产出，可直接放入参赛仓。

3.5 系统测试与结果分析

81. **测试环境：**硬件为 SF32LB52 开发板 + 自研板级配置，2.4 英寸 240×320 半透半反屏（NV3031A）、u-blox MAX-M10S + 外置天线、BMI270 与 BMP388、16 GB microSD；软件为 openvela（NuttX 内核）+ LVGL 9 + FreeType，产品固件版本 1.0.0、二级 boot 版本 1.0.4。测试方法包括三类：机上探针（nsh 下 30 余条自测命令，覆盖 SD、文件系统、屏幕恢复、传感器、蓝牙、看门狗等）、串口结构化日志与 diag 巡检、以及户外实骑验证。

82. **功能测试：**各项核心功能逐项验证，结果如下表。

测试项	实际结果
待机与双键翻页（7 页）	通过；短按翻页、长按菜单，转向页仅在导航时出现
开始 / 暂停 / 保存记录	通过；自动暂停 4 s、恢复 1.5 s，手动恢复后抑制 1 min
地图跟车、轨迹与公里标	通过；离开地图页仍继续采集，返回后补画轨迹不整图重绘
离线全国路网规划	通过；吸附 150 m，折线上限 2048 点
偏航重规划与到达判定	通过；偏航 35 m 重规划（最小间隔 8 s），到达 45 m 结束
沿 GPX 导航与返航	通过；窗口 8 km 续载，返航走廊放宽至 100 m
爬升剖面与坡度归零	通过；爬升 3 m 迟滞，坡度来自 IMU 并可水平归零
手机通知弹窗与收件箱	通过；按类型区分来电/短信/日历/应用，收件箱保留 8 条
USB MTP 拷贝地图与轨迹	通过；产品固件仅暴露骑行目录，工厂固件暴露三卷
BLE 文件传输与断点续传	通过；CRC32 校验，支持偏移续传

测试项	实际结果
OTA 升级与回退	通过；/fw 最多保留 12 个版本，回退 main → factory
工厂模式板卡测试	通过；充电 + 双键或引导连按 F 进入，逐页通过

83. **性能测试：**下表为实测数据（均为本机串口日志或机上探针测得）。

测试项	实测结果
LittleFS 挂载（512 MiB 卷）	0.474 s
挂载期读取扇区数	10 904 → 868
SD 顺序读吞吐	约 1.0 MB/s → 3.1 MB/s
SD 单块（512 B）读吞吐	约 0.76 MB/s
单扇区固定开销	439 / 495 / 644 µs（顺序 / 顺序 / 跨步）
SD 卡时钟	上限 24 MHz；失败可临时降 6 MHz，成功后必须回 24 MHz
USB MTP 大文件上传	18.9 MB 约 110 s（约 170 KB/s）
BLE 文件传输	约 15 KB/s；1 MB GPX 约 70 s
BLE MTU	请求 247，实测协商 193（有效载荷 190 B）
地图分片生成（PC 端）	70–204 tiles/s（并行 5 路）
地图产物规模（3 城）	5 925 个 .vpk，约 314 MB，489 898 条跨网格 portal
地图常驻内存（PSRAM）	分片缓存 1.5 MB + 包堆 1.0 MB + 轨迹 0.5 MB
渲染时间预算	每次定时器最多 2 个分片 / 8 ms，脏矩形最多 8 个

测试项	实测结果
显示通路	240×320 RGB565，4 线 QAD-SPI @24 MHz，60 行条带刷新
看门狗	IWDT 15 s 超时 / 2 s 喂狗 / 10 s 心跳 / 60 s UI 宽限
轨迹与记录容量	轨迹环约 5 万点（点距 3 m，约 150 km）；GPX 写入 1 Hz

84. **功耗说明（如实标注）：本机尚未接入电流表完成整机功耗标定，因此不在此处给出推测数值。当前的功耗设计目标是：背光关闭 + GNSS 待机 + Flash 断电的休眠状态，以及电池供电连续骑行不少于 10 小时（屏幕中等亮度、GNSS 1 Hz、不连蓝牙）。整机电流与续航的实测标定列入下一步计划。**

85. **可靠性与稳定性测试：**以“某一路失效只降级这一路、不整机复位”为原则，逐项验证。

86. · 看门狗：独立看门狗 15 s 超时，且要求空闲线程近期运行过且 UI 线程进过循环，因此正常情况下不会被喂狗掩盖卡死，只在整机真死时复位。

87. · 子系统巡检：diag 线程每 500 ms 巡检，按分级阈值（蓝牙 4/20 s、GNSS 12/30 s、DVFS 2/10 s、文件系统 3/15 s）判定健康度，对停顿线程主动踢活。

88. · GNSS 恢复：静默 5 s 重开 UART，连续两次不恢复则断电 2.5 s 重新上电；室内持续搜星时不做反复断电，避免越恢复越差。

89. · 存储恢复：SD I/O 超时先回 TRAN 状态再重试；控制器卡死时以模块级 RCC 复位恢复；插入 USB MTP 时地图让出存储，拔线后继续使用。

90. · 路由自愈：规划 worker 15 s 未响应判为卡死并自动重启，最多 3 次。

91. · 蓝牙重连：配件掉线后界面保留上次有效值，配对过的设备自动回连；空轮回连采用退避策略（窗口逐次加大至 32 s 上限），避免空转抢占射频。

92. · 异常兜底：显示通路具备超时→中止→恢复状态机；日志采用非阻塞环形缓冲，只落 WARNING 以上，保证故障瞬间不因写盘而雪崩。

93. **尚未标定的项目（不以推测值充数，列入下一步计划）：① 整机功耗（休眠电流、待机电流、骑行中平均电流）；② 10 h 以上连续运行与内存/帧率波动的长稳数据；③ GNSS 冷启动与热启动首次定位时间；④ 轨迹里程与手机 / 其他码表的对比误差。**

3.6 AI-Native 开发说明

94. 本作品全程采用 AI 结对开发：AI 承担驱动改写、地图工具链、协议实现与工程文档，人负责产品定义、硬件验证与最终取舍。下表为量化口径。

指标	数据
AI Coding 代码占比	约 85%（口径：以自研 C / Dart 代码约 25.7 万行为分母，其中由 AI 结对生成或改写的部分；人工负责产品定义、硬件验证与最终取舍）

指标	数据
使用的 AI 工具	ZCode（Z.ai），主力模型 DeepSeek-V4-Flash（本作品可复核的 12 139 次模型请求均记录为该模型）；配 mimosa 1.0.3 插件（钩子式代码审查与发现台账）。另使用 Cursor（部分提交带 Co-authored-by: Cursor）。
MCP 工具使用情况	使用 mimosa 1.0.3（MCP 插件）：提交前对整仓做项目级深度安全扫描共 3 次（输出含密封标识与可复核的发现台账）；未使用 VelaJS MCP、Figma MCP 等官方 MCP。
Skills 使用与新增情况	使用：skill-creator、document-skills 等第三方 Skill；新增沉淀 3 个：vela-helm-one（板级知识，见 3.4 节）、single-conversation-subagent（单对话两级协作方法论，独立仓开源）、codex-review（Codex 只读独立审查，本作品连审 5 轮 REJECT×3→ACCEPT×2）；另沉淀 AI 工具链：vela_my_vendor_tools.py（19 个子命令的单一入口）、serial_hub.py、pty_run.py、probe_boot.py
Token 使用总量	输入 5 069 545 267 / 输出 11 381 410（可由随仓 logs/ 的 JSONL 直接求和复核）。

95. **AI 工具对开发效率的提升：**本作品在约三个半月内由单人完成约 25.7 万行自研代码（386 个 C 文件）与 107 个 Dart 文件，覆盖从芯片驱动到 UI、从 PC 端地图工具链到手机 App 的完整链路。AI 主要承担四类工作：驱动改写（SD、LCD、GNSS、USB、蓝牙）、PC 端地图工具链（OSM 解析、分片打包、路由图构建）、双端协议实现，以及工程文档的整理与一致性维护。

96. **AI 使用规模（可复核）：本作品相关会话 34 个（9 个主会话 + 25 个子代理）、用户轮次 1 013、AI 工具调用 12 336 次（Bash 8 298 / Edit 2 146 / Read 1 509 / Write 229 / Agent 25）、AI 侧轮次耗时累计 15.7 小时。本机留存日志覆盖 2026-09-16 ~ 09-20；更早阶段使用其他工具（如 Cursor），故「三个半月」是整体开发周期，与此处可复核口径不等长。**

97. **三个真实案例：**

98. · SD 控制器死锁：现象是整卡假死、1 Hz 反复重识别。AI 在读控制器状态与官方参考实现后，定位到控制器卡在 CMD_BUSY，给出模块级 RCC 复位方案，问题根治。

99. · LittleFS 挂载 0.474 s：AI 通过统计挂载期实际读取的扇区数（10 904 个）定位到缓存与预读参数过大，调整后读取量降到 868 个扇区。

100. · 蓝牙双角色冲突：AI 直接读 zblue 源码，定位到 LCPU 在已有连接时禁止设置随机地址并返回 -EACCES，据此产出上游补丁，实现“连手机的同时扫描传感器”。

101. **遇到的问题与解决方式：**最大的风险是 AI 会给出看似合理但未经硬件验证的改动。为此建立了三层验证：机上探针（nsh 自测命令直接读寄存器与外设状态）、串口结构化日志、以及 diag 巡检；所有结论以真机实测为准。此外引入钩子式代码审查（mimosa）在提交前检查改动，并把每次踩坑的原因与处置写入仓库文档，形成可复用的记忆。

3.7 总结与展望

102. **成果总结：**本作品完成了一台可上路的开源骑行码表：从芯片驱动、板级适配、离线地图工具链、整车 UI 到手机 App 的全链路自研。核心指标上，LittleFS 挂载 0.474 s，SD 顺序读提升到 3.1 MB/s，三座城市的离线地图产物为 5 925 个分片、约 314 MB，并在此基础上实现了全国离线路网规划、GPX 轨迹导航与返航、通知转发、USB MTP 与 OTA。系统、板级、应用、App 与地图工具链全部开源（Apache 2.0）。

103. **应用前景与商业价值：**目标受众是公路与通勤骑行者（以 25–45 岁、一二线城市、有开源与可维修偏好的骑行爱好者为主），以及希望在 openvela 上做运动穿戴的开发者。产品的差异化在于“完全离线 + 无触摸双键 + 开源”，既避开了手机支架耗电与无网失效的问题，也避开了成品码表闭源高价的定位。商业模式上，开源可用于建立社区与开发者生态，整机与工厂灌图服务面向成品市场，地图包更新与 App 增值可作为持续性收入。同一板级与地图引擎可横向复用到跑步表、摩托车码表与户外仪表，具备规模化摊薄研发成本的潜力。

104. **不足与未来工作：**① 当前硬件为自研 PCB 与自研外壳的 1.0 版，量产版的结构、装配工艺与成本迭代尚未完成；② 整机功耗与连续续航尚未完成实测标定；③ 产品目前没有端侧 AI 能力，后续可考虑加入骑行姿态识别、路面识别或语音唤醒（如加入语音，将按大赛要求统一使用“你好，openvela”唤醒词）；④ BLE 传文件速率约 15 KB/s，大文件仍需依赖 USB MTP；⑤ 传感器绑定尚未使用 IRK，使用随机地址的配件需要固定地址才能稳定回连，后续补齐。

三、评审维度对照

评审维度（分值）	报告对应章节
技术难度（30）	3.2 系统方案设计；3.3 核心算法与技术原理；3.4 系统实现；3.5 系统测试
产品创新性（20）	2、摘要；3.1 绪论（创新点）
项目完整度（20）	3.5 系统测试与结果分析；项目源码；作品展示照片
AI 开发（10）	3.3 核心算法（AI 算法实现）；3.6 AI-Native 开发说明
商业潜力（10）	3.7 总结与展望（应用前景与商业价值）
展示效果（10）	演示视频；海报；答辩 PPT